

Projeto e Análise de Algoritmos (2026/1)

Projeto de Formalização

Flávio L. C. de Moura¹

flaviomoura@unb.br

4 de junho de 2026

O presente projeto tem como objetivo a construção de uma formalização em um assistente de provas sobre algum tema ligado à disciplina de Projeto e Análise de Algoritmos. O projeto pode ser desenvolvido individualmente, ou em grupo de até 3 alunos. A seguir listamos algumas propostas. Cada grupo deve selecionar uma proposta, mas a seleção de mais de uma proposta também é possível. A ferramenta sugerida para desenvolvimento do projeto é o assistente de provas Rocq (<https://rocq-prover.org>), mas outros assistentes de prova podem ser utilizados conforme preferência do grupo. Outras opções incluem o Lean (<https://lean-lang.org/>) e o PVS (<https://sri-fm.github.io/pvs/>). Ao final de cada proposta existe um link para um repositório que pode ser clonado e utilizado como ponto de partida do projeto, caso o grupo opte por fazer o projeto no assistente de provas Rocq.

1 A correção do algoritmo Bubble Sort

Este trabalho apresenta uma prova formal da correção do algoritmo de ordenação por borbulhamento (a função *bs* a seguir). A formalização foi feita no assistente de provas Coq. O assistente de provas Coq utiliza o sistema de Dedução Natural, o que o torna adequado para o desenvolvimento de atividades computacionais no curso de Lógica Computacional 1. O Coq permite a extração de código certificado em diversas linguagens funcionais, como Ocaml, Haskell e Scheme.

Iniciaremos definindo a função *bubble* que recebe uma lista de naturais como argumento, e percorre esta lista comparando elementos consecutivos. Chamamos este processo de borbulhamento:

```
Function bubble (l: list nat) {measure length l} :=
  match l with
  | nil => nil
  | x::nil => x::nil
  | x::y::l =>
    if x <=? y
    then x::(bubble (y::l))
    else y::(bubble (x::l))
  end.
```

Observe que esta função não é estruturalmente recursiva porque, por exemplo, a lista $(x::l)$ não é uma sublista da lista original $(x::y::l)$. Neste caso, utilizamos **Function** para construir esta função e precisamos fornecer a medida que decresce em cada chamada recursiva, além de provar que esta medida efetivamente decresce a cada chamada recursiva. Por exemplo, *bubble* $(2::1::nil)$ retorna a lista $(1::2::nil)$.

Eval compute in *bubble* $(2::1::nil)$.

```
= 1 :: 2 :: nil
: list nat
```

Eval compute in *bubble* $(3::2::1::nil)$.

```
= 2 :: 1 :: 3 :: nil
: list nat
```

A função principal, ou seja, o algoritmo bubble sort propriamente dito, é dada pela função *bs* abaixo que recebe uma lista de naturais como argumento:

```
Fixpoint bs (l: list nat) :=
  match l with
  | nil ⇒ nil
  | h::l' ⇒ bubble (h::(bs l'))
  end.
```

Sabemos que aplicar a função *bubble* a uma lista qualquer, não necessariamente vai retornar uma lista ordenada, mas o lema *bubble_sorted* a seguir nos mostra que se o primeiro elemento é o único elemento fora de ordem em uma lista, ao aplicarmos a função *bubble*, obtemos uma lista ordenada:

Lemma *bubble_sorted*: $\forall l, \text{Sorted } l \rightarrow \text{bubble } l = l$.

Lemma *bs_sorted*: $\forall l, \text{Sorted } l \rightarrow \text{bs } l = l$.

A seguir, mostraremos que o algoritmo bubblesort (função *bs*) gera como saída uma permutação da lista de entrada. O lema a seguir nos diz que a função *bubble* também gera uma permutação da entrada:

Lemma *bubble_perm*: $\forall l, \text{Permutation } l (\text{bubble } l)$.

O lema *bs_correto* a seguir, nos mostra que o algoritmo *bs* gera uma permutação da lista de entrada:

Lemma *bs_permuta*: $\forall l, \text{Permutation } l (\text{bs } l)$.

Por fim, a correção do algoritmo *bs* é obtida pelo teorema a seguir que estabelece que o algoritmo *bs* retorna uma permutação da lista de entrada que está ordenada:

Theorem *bs_correto*: $\forall l, \text{Sorted } l \rightarrow (\text{bs } l) \wedge \text{Permutation } l (\text{bs } l)$.

Repositório: https://github.com/flaviodemoura/bubble_sort

2 A correção do algoritmo Selection Sort

A função *select_min* a seguir, recebe uma lista de naturais e retorna o menor elemento desta lista. Se a lista for vazia, *select_min nil* retorna *None*.

```
Function select_min (l: list nat) {measure length l} : option nat :=
  match l with
  | nil ⇒ None
  | h::nil ⇒ Some h
  | h1::h2::tl ⇒ if h1 <=? h2 then select_min (h1::tl) else select_min (h2::tl)
  end.
```

Eval compute in (*select_min* (1::2::3::nil)).

Eval compute in (*select_min* (4::2::1::3::nil)).

Eval compute in (*select_min* (40::23::111::3::nil)).

Definition *le_all* $x \ l := \forall y, \text{In } y \ l \rightarrow x \leq y$.

A correção da função *select_min* é estabelecida provando-se que, se *select_min l* retorna um natural *m* então *m* é menor ou igual do que todos os elementos de *l*.

Lemma *select_min_correct* : $\forall l \ m, \text{select_min } l = \text{Some } m \rightarrow \text{le_all } m \ l$.

A função principal *ss* recebe uma lista de naturais *l*, e retorna uma permutação ordenada de *l*: **Fixpoint** *ss* (*l*: *list nat*) :=

```

match l with
| nil  $\Rightarrow$  nil
| h::tl  $\Rightarrow$  match select_min l with
    | None  $\Rightarrow$  nil
    | Some m  $\Rightarrow$  m::(ss tl)
end
end.

```

A correção do algoritmo *ss* é obtida a partir da prova de que *ss* retorna uma permutação ordenada da lista de entrada.

Theorem *selectionsort_correct*: $\forall l, \text{Sorted } le \ (ss \ l) \wedge \text{Permutation } l \ (ss \ l)$

Repositório: https://github.com/flaviodemoura/selection_sort

3 A correção do algoritmo Mergesort

Neste trabalho formalizaremos a correção do algoritmo *mergesort*. Esta formalização envolve diversas etapas que incluem a definição de diferentes funções.

O algoritmo *merge* a seguir recebe um par de listas ordenadas como argumento. A função *len* abaixo, define o tamanho de um par de listas:

Definition *len* (*p*:*list nat* $\times$ *list nat*) := *length* (*fst p*) + *length* (*snd p*).

Function *merge* (*p*: *list nat* $\times$ *list nat*) {**measure** *len p*} :=

```

match p with
| ([], l2)  $\Rightarrow$  l2
| (l1, [])  $\Rightarrow$  l1
| (h1::l1, h2::l2)  $\Rightarrow$ 
    if h1 <=? h2
    then h1::(merge (l1,h2::l2))
    else h2::(merge (h1::l1,l2))
    end.

```

A seguir apresentamos algumas definições e lemas que podem ser úteis. Eles podem ser modificados ou removidos de acordo com a sua estratégia de prova. Outros resultados auxiliares podem ser adicionados, se necessário.

Definition *le_all* *x l* := $\forall y, \text{In } y \ l \rightarrow x \leq y$.

Lemma *le_all_sorted*: $\forall l \ x, \text{Sorted } le \ l \rightarrow le_all \ x \ l \rightarrow \text{Sorted } le \ (x::l)$.

Lemma *sorted_le_all*: $\forall l \ x, \text{Sorted } le \ (x::l) \rightarrow le_all \ x \ l$.

Lemma *merge_permuta*: $\forall (l1 \ l2: \text{list nat}), \text{Permutation } (l1 ++ l2) \ (merge(l1,l2))$.

Lemma *merge_correto*: $\forall l1 \ l2, \text{Sorted } le \ l1 \rightarrow \text{Sorted } le \ l2 \rightarrow \text{Sorted } le \ (merge \ (l1,l2))$.

O algoritmo *mergesort* é definido como a seguir:

Function *mergesort* (*l*: *list nat*) {**measure** *length l*} :=

```

match l with

```

```

| [] ⇒ []
| [h] ⇒ [h]
| h1::h2::l' ⇒
  let l1_half := Nat.div2 (length l) in
  let l1 := firstn l1_half l in
  let l2 := skipn l1_half l in
  merge(mergesort l1 , mergesort l2)
end.

```

A correção do algoritmo *mergesort* é obtida com a prova do teorema abaixo:

Theorem *mergesort_correto*: $\forall l, \text{Sorted } le \text{ (mergesort } l) \wedge \text{Permutation } l \text{ (mergesort } l)$.

Repositório: https://github.com/flaviodemoura/merge_sort

4 A correção do algoritmo de ordenação por inserção binária

A função *bsearch* x l retorna a posição i ($0 \leq i \leq |l| - 1$) onde x deve ser inserido na lista ordenada l :

```

Function bsearch x l {measure length l} :=
  match l with
  | [] ⇒ 0
  | [y] ⇒ if (x <=? y)
    then 0
    else 1
  | h1::h2::tl ⇒
    let len := length l in
    let mid := len / 2 in
    let l1 := firstn mid l in
    let l2 := skipn mid l in
    match l2 with
    | [] ⇒ 0
    | h2'::l2' ⇒ if (x <? h2')
      then bsearch x l1
      else
        if x =? h2'
        then mid
        else mid + (bsearch x l2)
    end
  end
end.

```

Podemos fazer alguns testes com esta função:

Lemma *test0*: *bsearch* 1 [] = 0.

Lemma *test1*: *bsearch* 1 [0;2;3] = 1.

Lemma *test2*: *bsearch* 2 [0;2;3] = 1.

Lemma *test3*: *bsearch* 2 [0;2;3;4] = 1.

Lemma *test4*: $bsearch\ 3\ [0;1;2;3;4;5] = 3$.

Também podemos verificar que $bsearch\ x\ l$ sempre retorna uma posição válida da lista l :

Lemma *bsearch_valid_pos*: $\forall l\ x, 0 \leq bsearch\ x\ l < length\ l$.

A seguir, definiremos a função $insert_at\ i\ x\ l$ que insere o elemento x na posição i da lista l :

```

Fixpoint insert_at  $i\ (x:nat)\ l :=$ 
  match  $i$  with
  |  $0 \Rightarrow x::l$ 
  |  $S\ k \Rightarrow$  match  $l$  with
    |  $nil \Rightarrow [x]$ 
    |  $h::tl \Rightarrow h::(insert\_at\ k\ x\ tl)$ 
  end
end.

```

Eval **compute** **in** $(insert_at\ 2\ 3\ [1;2;3])$.

A função $binsert\ x\ l$ a seguir insere o elemento x na posição retornada por $bsearch\ x\ l$ de l :

```

Definition binsert  $x\ l :=$ 
  let  $pos := bsearch\ x\ l$  in
   $insert\_at\ pos\ x\ l$ .

```

Agora podemos enunciar o teorema que caracteriza a correção da função $binsert$, a saber, que $binsert\ x\ l$ retorna uma lista ordenada, se l estiver ordenada:

Theorem *binsert_correct*: $\forall l\ x, Sorted\ le\ l \rightarrow Sorted\ le\ (binsert\ x\ l)$.

Alternativamente, podemos construir uma única função que combina a execução de $bsearch$ e $insert_at$. A função $binsert\ x\ l$ a seguir, recebe o elemento x e a lista ordenada l como argumentos e retorna uma permutação ordenada da lista $x::l$:

```

Function binsert'  $x\ l\ \{\text{measure } length\ l\} :=$ 
  match  $l$  with
  |  $[] \Rightarrow [x]$ 
  |  $[y] \Rightarrow$  if  $(x <=?\ y)$ 
    then  $[x; y]$ 
    else  $[y; x]$ 
  |  $h1::h2::tl \Rightarrow$ 
    let  $len := length\ l$  in
    let  $mid := len / 2$  in
    let  $l1 := firstn\ mid\ l$  in
    let  $l2 := skipn\ mid\ l$  in
    match  $l2$  with
    |  $[] \Rightarrow l$ 
    |  $h2'::l2' \Rightarrow$  if  $x =?\ h2'$ 
      then  $l1 ++ (x::l2)$ 
      else
        if  $x <?\ h2'$ 
        then  $binsert'\ x\ l1$ 

```

```

      else binsert' x l2
    end
  end.

```

Lemma *teste0*: (*binsert'* 2 [1;2;3]) = [1;2;2;3].

As funções *binsert* e *binsert'* realizam o mesmo trabalho:

Lemma *binsert_equiv_binsert'*: $\forall l x, \text{binsert } x l = \text{binsert}' x l$.

E portanto a correção de *binsert'* é imediata:

Corollary *binsert'_correct*: $\forall l x, \text{Sorted } le l \rightarrow \text{Sorted } le (\text{binsert}' x l)$.

O algoritmo principal é dado a seguir:

```

Fixpoint bininsertion_sort (l: list nat) :=
  match l with
  | [] => []
  | h::tl => binsert h (bininsertion_sort tl)
  end.

```

O teorema a seguir caracteriza a correção do algoritmo *bininsertion_sort*. Observe que pode ser conveniente dividir esta prova em outras provas menores. Isto significa que a formalização pode ficar mais simples e mais organizada com a inclusão de novos lemas.

Theorem *bininsertion_sort_correct*: $\forall l, \text{Sorted } le (\text{bininsertion_sort } l) \wedge \text{Permutation } l (\text{bininsertion_sort } l)$.

Repositório: https://github.com/flaviodemoura/bininsertion_sort

5 Equivalência entre diferentes noções de indução

Seja P uma propriedade sobre os números naturais. O Princípio da Indução Matemática (PIM) pode ser enunciado da seguinte forma:

Definition *PIM* :=

$$\begin{aligned}
 &\forall P: \text{nat} \rightarrow \text{Prop}, \\
 & (P \ 0) \rightarrow \\
 & (\forall k, P \ k \rightarrow P \ (S \ k)) \rightarrow \\
 & \forall n, P \ n.
 \end{aligned}$$

Seja Q uma propriedade sobre os números naturais. O Princípio da Indução Forte (PIF) pode ser enunciado da seguinte forma:

Definition *PIF* :=

$$\begin{aligned}
 &\forall Q: \text{nat} \rightarrow \text{Prop}, \\
 & (\forall k, (\forall m, m < k \rightarrow Q \ m) \rightarrow Q \ k) \rightarrow \\
 & \forall n, Q \ n.
 \end{aligned}$$

Dado um predicado P sobre naturais, se existe um natural n que satisfaz a propriedade P , então existe um m que é o menor natural que satisfaz a propriedade P . Esta propriedade é conhecida como o Princípio da Boa Ordenação (PBO):

Definition *PBO* := $\forall P : \text{nat} \rightarrow \text{Prop},$
 $(\exists n : \text{nat}, P \ n) \rightarrow$
 $\exists m : \text{nat}, P \ m \wedge \forall x : \text{nat}, x < m \rightarrow \neg P \ x.$

Prove que estes princípios são equivalentes:

Theorem *PIM_equiv_PIF*: $PIM \leftrightarrow PIF$.

Theorem *PBO_equiv_PIM*: $PBO \leftrightarrow PIM$.

Theorem *PBO_equiv_PIF*: $PBO \leftrightarrow PIF$.

Repositório: https://github.com/flaviodemoura/ind_equiv

6 Equivalência entre diferentes noções de ordenação

Equivalência entre diferentes noções de ordenação.

A primeira definição de ordenação, chamada *ord1*, é uma definição indutiva contendo 3 regras de formação:

$$\frac{}{ord1\ nil} (ord1_nil) \quad \frac{}{ord1\ (x :: nil)} (ord1_one) \quad \frac{x \leq y \quad ord1(y :: l)}{ord1\ (x :: y :: l)} (ord1_all)$$

Inductive *ord1* : *list nat* → Prop :=

| *ord1_nil*: *ord1 nil*

| *ord1_one*: $\forall x, ord1\ (x :: nil)$

| *ord1_all*: $\forall l\ x\ y, x \leq y \rightarrow ord1\ (y :: l) \rightarrow ord1\ (x :: y :: l)$.

A segunda definição de ordenação, chamada *ord2*, é uma definição indutiva contendo 2 regras de formação:

$$\frac{}{ord2\ nil} (ord2_nil) \quad \frac{x \leq^* l \quad ord2\ l}{ord2\ (x :: l)} (ord2_all)$$

onde $x \leq^* l$ significa que x é menor ou igual que todo elemento da lista l . Formalmente, este predicado é definido como a seguir:

Definition *le_all* $x\ l := \forall y, In\ y\ l \rightarrow x \leq y$.

Inductive *ord2* : *list nat* → Prop :=

| *ord2_nil*: *ord2 nil*

| *ord2_all*: $\forall l\ x, x \leq^* l \rightarrow ord2\ l \rightarrow ord2\ (x :: l)$.

A terceira definição de ordenação, chamada *ord3*, diz que uma lista está ordenada se cada elemento é menor ou igual ao elemento seguinte:

Definition *ord3* $(l : list\ nat) : Prop := \forall i, length\ l > 1 \rightarrow (S\ i) < length\ l \rightarrow nth\ i\ l\ 0 \leq nth\ (S\ i)\ l\ 0$.

Lemma *ord3_nil*: *ord3 nil*.

Lemma *ord3_one*: $\forall x, ord3\ (x :: nil)$.

Lemma *ord3_two*: *ord3* $(1 :: 2 :: nil)$.

A quarta definição de ordenação, chamada *ord4*, diz que uma lista está ordenada se cada elemento é menor ou igual que qualquer elemento que esteja em posição anterior:

Definition *ord4* $(l : list\ nat) : Prop := \forall i\ j, i < j \rightarrow j < length\ l \rightarrow nth\ i\ l\ 0 \leq nth\ j\ l\ 0$.

Lemma *ord4_nil*: *ord4 nil*.

O objetivo desta proposta é mostrar que as definições *ord1*, *ord2*, *ord3* e *ord4* são equivalentes:

Theorem *ord1_equiv_ord2*: $\forall l, \text{ord1 } l \leftrightarrow \text{ord2 } l$.

Theorem *ord1_equiv_ord3*: $\forall l, \text{ord1 } l \leftrightarrow \text{ord3 } l$.

Theorem *ord1_equiv_ord4*: $\forall l, \text{ord1 } l \leftrightarrow \text{ord4 } l$.

Theorem *ord2_equiv_ord3*: $\forall l, \text{ord2 } l \leftrightarrow \text{ord3 } l$.

Theorem *ord2_equiv_ord4*: $\forall l, \text{ord2 } l \leftrightarrow \text{ord4 } l$.

Theorem *ord3_equiv_ord4*: $\forall l, \text{ord3 } l \leftrightarrow \text{ord4 } l$.

Repositório: https://github.com/flaviodemoura/ord_equiv

7 Equivalência entre diferentes noções de permutação

onde o predicado *Permutation* é definido pelas regras a seguir:

$$\frac{}{\text{Permutation nil nil}} \text{ (perm_nil)} \quad \frac{\text{Permutation } l \ l'}{\text{Permutation } (x :: l) \ (x :: l')} \text{ (perm_skip)}$$

$$\frac{}{\text{Permutation } (y :: x :: l) \ (x :: y :: l)} \text{ (perm_swap)}$$

$$\frac{\text{Permutation } l \ l' \quad \text{Permutation } l' \ l''}{\text{Permutation } l \ l''} \text{ (perm_trans)}$$

onde x, y, l, l' e l'' são variáveis universais.

O código Coq correspondente a essas regras é listado a seguir:

```
Inductive Permutation (A : Type) : list A -> list A -> Prop :=
| perm_nil : Permutation nil nil
| perm_skip : forall (x : A) (l l' : list A),
    Permutation l l' ->
    Permutation (x :: l) (x :: l')
| perm_swap : forall (x y : A) (l : list A),
    Permutation (y :: x :: l)
    (x :: y :: l)
| perm_trans : forall l l' l'' : list A,
    Permutation l l' ->
    Permutation l' l'' ->
    Permutation l l''.
```

Alternativamente, a lista l é uma permutação da lista l' se ambas possuem os mesmos elementos. A implementação dessa ideia é feita baseada no número de ocorrência de cada elemento na lista. Assim, na definição a seguir, *num_oc x l* retorna o número de ocorrências do elemento x na lista l :

```

Fixpoint num_oc x l :=
  match l with
  | nil => 0
  | h :: tl =>
    if x =? h then S(num_oc x tl) else num_oc x tl
  end.

```

A definição *equiv* a seguir, expressa que a lista l é uma permutação da lista l' se ambas possuem o mesmo número de ocorrências de qualquer elemento:

Definition *equiv* $l\ l' := \forall n:nat, \text{num_oc } n\ l = \text{num_oc } n\ l'$.

Lemma *perm_to_equiv*: $\forall l\ l', \text{Permutation } l\ l' \rightarrow \text{equiv } l\ l'$.

Lemma *equiv_to_perm*: $\forall l\ l', \text{equiv } l\ l' \rightarrow \text{Permutation } l\ l'$.

O teorema a seguir formaliza que as definições *Permutation* e *equiv* são equivalentes, e portanto qualquer uma delas pode ser utilizada no projeto.

Theorem *perm_equiv*: $\forall l\ l', \text{Permutation } l\ l' \leftrightarrow \text{equiv } l\ l'$.

Repositório: https://github.com/flaviodemoura/perm_equiv

8 Tema livre

Alternativamente, os grupos podem trabalhar com outros algoritmos e/ou teorias desde que não sejam excessivamente simples. Consulte o professor para que a escolha seja aprovada, e uma estratégia de prova seja construída.

9 Etapas do Projeto

- O projeto deve ser desenvolvido em um repositório web (algo como Github ou GitLab), e o link deve ser enviado via chat do Teams até o dia [2026-06-10 qua]. Qualquer mudança ou informação relevante em relação ao desenvolvimento do projeto deve ser atualizada na mesma conversa do Teams para permitir uma comunicação mais organizada com o grupo. A qualquer momento o professor pode fazer sondagens (presenciais ou online) sobre o desenvolvimento do projeto.
- O prazo para finalização do projeto é o dia [2026-07-08 qua]
- **Apresentação do projeto:** As apresentações dos projetos (presenciais ou remotas síncronas) poderão ser agendadas para os dias [2026-07-08 qua], [2026-07-13 seg] ou [2026-07-15 qua]. No caso de apresentação remota assíncrona (gravação de vídeo), o link para o mesmo deverá ser disponibilizado juntamente com a entrega do projeto no dia [2026-07-08 qua]
- **Uso de Inteligência Artificial:** Os grupos podem utilizar ferramentas de IA (como assistentes de código, modelos de linguagem, etc.) no desenvolvimento do

projeto, desde que de forma **crítica e reflexiva**. O professor pode consultar todos os membros do grupo individualmente para verificar o entendimento e a participação de cada um no trabalho. O uso de IA não desobriga nenhum integrante de compreender e saber explicar qualquer parte do projeto.

- **Erros nos temas propostos:** Os temas propostos podem conter pequenos erros ou imprecisões. A identificação e a correção desses erros **será valorizada positivamente** na avaliação do projeto e deve estar documentada no relatório com a devida justificativa.
- **Migração de tema:** Os grupos podem mudar de tema durante o desenvolvimento do projeto. Neste caso, os resultados parciais obtidos no tema anterior **devem ser documentados no relatório**, explicando os motivos da migração e o que foi aproveitado ou aprendido.
- **Adaptações nos códigos:** Os grupos podem modificar e ajustar os códigos fornecidos como ponto de partida sempre que julgarem necessário. Toda modificação deve estar **bem explicada no relatório**, com motivação e justificativa claras para as escolhas realizadas.
- O repositório deve conter:
 1. O(s) arquivo(s) com as provas como descrito no relatório.
 2. **Relatório escrito (opcional):** Consiste em um arquivo em formato **pdf** contendo o **nome e matrícula dos participantes**, assim como todas as informações relevantes sobre o desenvolvimento do projeto.